Pregunta 1

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

Which of the following lines of code correctly appends a string to a StringBuilder object in Java?

Seleccione una:

a. StringBuilder sb = new StringBuilder("Hello"); sb.add(" World");

b. StringBuilder sb = new StringBuilder("Hello"); sb.append(" World"); ✓

c. StringBuilder sb = "Hello"; sb += " World";

d. StringBuilder sb = new StringBuilder("Hello"); sb.insert(" World");

Justificación: append() añade contenido al final del StringBuilder. add() no existe. Un literal String no se puede asignar a un StringBuilder, ni += está definido para StringBuilder. insert() sí existe, pero su primer argumento es el índice, así que insert(" World") no compila.

Fuente: Java.pdf #190

La respuesta correcta es: StringBuilder sb = new StringBuilder("Hello"); sb.append(" World");

Mi justificación
XXX a) El método add() no es de StringBuilder, c) No se puede concatenar Stringbuider con +=, d) Aunque insert() si es método de StringBuilder, requiere un primer parámetro de índice dónde se insertará el texto. El único correcto es b)

Pregunta 2

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

¿Cuál de las siguientes líneas de código utiliza correctamente un método de la clase String para reemplazar todas las apariciones del carácter 'a' por 'e' en una cadena dada en Java?

Seleccione una:

- a. String result = "banana".replace('a' ,'e'); ✓
- b. String result = "banana".change('a' ,'e');
- c. String result ="banana".modify('a' ,'e');
- d. String result = "banana".substitute('a' ,'e');
- e. String result = "banana".swap('a' ,'e');

Justificación: String.replace(char, char) devuelve una cadena nueva con todas las apariciones del carácter reemplazadas: "benene". change, modify, substitute y swap no existen en String y no compilan.

Fuente: Simulación APX V2 #6 · también aparece en: Java.pdf #72

La respuesta correcta es: String result = "banana".replace('a' ,'e');

Mi justificación
El único correcto es replace ya que los demás no son métodos de String.

Pregunta 3

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

What is the best way to test that the values of h1 and h2 are the same?

```
public static void main(String[] args){
    String h1 = "Bob";
    String h2 = new String("Bob");
}
```

Seleccione una:

- a. if (h1.equals(h2)). ✓
- b. if (h1 == h2).
- c. if (h1.toString() == h2.toString()).
- d. if (h1.same(h2)).

Justificación: equals() compara el contenido de los String, que es igual. == compara referencias, y h2 es un objeto nuevo distinto del literal del pool, así que da false. toString() de un String devuelve la misma referencia, así que la opción c tampoco sirve, y same() no existe.

Fuente: Java D #18 · también aparece en: Quiz Desarrollo APX 1 #4

La respuesta correcta es: if (h1.equals(h2)).

Mi justificación
XXX b) No es válido ya que == compara las referencias de los objetos y no su valor, c) Lo mismo, d) El método same() no existe para Strings. VVV La única correcta es la a) ya que equals compara los values.

Pregunta 4

Correcta

Se puntúa 1.00 sobre 1.00

Marcar pregunta

Nivel: **Básico**

What is the result?

```
public class X {
    public static void main(String[] args) {
        String theString = "Hello World";
        System.out.println(theString.charAt(11));
    }
}
```

Seleccione una:

- a. There is no output.
- b. d is output.
- c. A StringIndexOutOfBoundsException is thrown at runtime. ✓
- d. An ArrayIndexOutOfBoundsException is thrown at runtime.
- e. A NullPointException is thrown at runtime.
- f. A StringArrayIndexOutOfBoundsException is thrown at runtime.

Justificación: "Hello World" tiene 11 caracteres (índices 0 a 10), así que charAt(11) lanza StringIndexOutOfBoundsException. El distractor d confunde el String con un arreglo, y StringArrayIndexOutOfBoundsException no existe.

Fuente: Java Sesiones #53 · también aparece en: Java D #21

La respuesta correcta es: A StringIndexOutOfBoundsException is thrown at runtime.

Mi justificación
El string “Hello World” tiene 11 caracteres, ósea que sus índices van del 0 al 10. Entonces charAt(11) daría un error por estar fuera del rango permitido del string.

Pregunta 5

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

What is the result?

```
public class Test {
    public void op2(String s) {
        s.toUpperCase();
    }
    public void op3() {
        String msg = new String("message");
        op2(msg);
        System.out.println(msg);
    }
    public static void main(String[] args) {
        new Test().op3();
    }
}
```

Seleccione una:

- a. MESSAGE
- b. An exception is thrown at runtime.
- c. Compilation fails.
- d. message ✓
- e. null

Justificación: String es inmutable: toUpperCase() devuelve una cadena nueva que op2 descarta. Además, Java pasa la referencia por valor, así que msg sigue apuntando a "message". El distractor MESSAGE supone que el método modifica el objeto original.

Fuente: Java.pdf #92 · también aparece en: Simulación APX V4 #30

La respuesta correcta es: message

Mi justificación
El comando s.toUpperCase() no se asignó a ninguna variable, y no altera el valor original de s así que se imprime como su valor original "message"

Pregunta 6

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

What is the result?

```
public class Test {
    public static void main(String[] args) {
        String a = "tic";
        String b = "tac";
        a.substring(0, 2);
        a.concat("toc");
        System.out.println(a += "game" + b.toUpperCase());
    }
}
```

Seleccione una:

- a. ticgameTAC ✓
- b. titocTAC
- c. tictactoc
- d. ticgametac
- e. Compilation fails.

Justificación: substring y concat devuelven cadenas nuevas que no se asignan a nada, así que a sigue siendo "tic". Luego a += "game" + "TAC" da "ticgameTAC", y la asignación compuesta es una expresión válida dentro de println. La d falla porque lo que se concatena es el valor devuelto por b.toUpperCase(), "TAC".

Fuente: Java.pdf #108 · también aparece en: Simulación APX V4 #37

La respuesta correcta es: ticgameTAC

Mi justificación
Aquí no hay truco, leyendo el print obtenemos “tic” += “game” + “TAC” lo que nos da ticgameTAC.

Pregunta 7

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

What will be the output of the following code snippet?

```
public class StringConcatenationTest {
    public static void main(String[] args) {
        String str1 = "Hello";
        String str2 = "World";
        String str3 = str1 + " " + str2;
        String str4 = str1.concat(" ").concat(str2);
        String str5 = new StringBuilder().append(str1).append(" ").append(str2).toString();
        System.out.print(str3.equals(str4) + " ");
        System.out.print(str4.equals(str5) + " ");
        System.out.print((str3 == str4) + " ");
        System.out.println(str4 == str5);
    }
}
```

Seleccione una:

- a. true true true true
- b. Compilation fails.
- c. true true false false ✓
- d. false false false false
- e. false true false true
- f. false false true true

Justificación: equals compara el contenido: las tres cadenas valen "Hello World", así que las dos primeras comparaciones dan true. == compara referencias: str3 (concatenación de variables en tiempo de ejecución), str4 (concat) y str5 (StringBuilder.toString) son objetos distintos y no internados, así que las dos comparaciones con == dan false.

Fuente: Java.pdf #89 · también aparece en: Simulación APX V4 #29

La respuesta correcta es: true true false false

Mi justificación
El primer print usa equals para comparar los values de str3 y str4, ambos son “Hello World”. Da true. El segundo print compara los values de str4 y str5, ambos son “Hello World”. Da true. El tercer print compara dos referencias de str3 y str4, y estos se construyeron como dos strings diferentes. Da false. El cuarto print compara dos referencias de str4 y str5, y estos se construyeron como dos strings diferentes. Da false

Pregunta 8

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

Que se imprime en pantalla al ejecutar el siguiente código?

```
public class StringInternExample {
    public static void main(String[] args) {
        String str1 = new String("example");
        String str2 = "example";
        String str3 = str1.intern();
        System.out.println(str1 == str2);
        System.out.println(str2 == str3);
        System.out.println(str1 == str3);
    }
}
```

Seleccione una:

- a. false true false
- b. true true true
- c. false false false ✗
- d. true false false

Justificación: new String crea un objeto distinto en el heap, así que str1 == str2 es false. intern() devuelve la instancia del pool, que es la misma del literal, así que str2 == str3 es true. str1 sigue siendo el objeto del heap, así que str1 == str3 es false. El distractor «true true true» confunde == con equals.

Fuente: Simulación APX V4 #6 · también aparece en: Simulación APX V1 #39, Java.pdf #17

La respuesta correcta es: false true false

Mi justificación
Los 3 strings se construyeron como strings diferentes así que todas las comperaciones con == dan false.

Pregunta 9

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

Given the following code snippet, what will be the output of the program?

```
String s1 = "text";
String s2 = new String("text");
String s3 = "text";
String s4 = s2.intern();
System.out.println(s1 == s3);
System.out.println(s1 == s2);
System.out.println(s1 == s4);
```

Seleccione una:

a. false false true

b. true true true

c. true false false ⊗

d. true false true

Justificación: s1 y s3 son el mismo literal del String pool (true). new String crea otro objeto en el heap (false). intern() devuelve la instancia canónica del pool, la misma que s1 (true). El distractor c olvida que intern() devuelve la referencia del pool.

Fuente: Java.pdf #129 · también aparece en: Cuestionario APX V3 #16

La respuesta correcta es: true false true

Mi justificación
Solo s1 y s3 dan true cuando se comparan con == ya que apuntan al mismo value y están en el mismo pool.

Pregunta 10

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

¿Qué imprime en pantalla este código?

```
public class Test {
    public static void main(String[] args) {
        String str = "a1b2c3d4e5f6";
        String[] parts = str.split("\\D");
        for (String part : parts) {
            System.out.print(part + " ");
        }
    }
}
```

Seleccione una:

a. 1 2 3 4 5 6
b. Compilation fails.
c. abcdef
d. An exception is thrown at runtime.
e. 123456 ⊗

Justificación: \D coincide con cualquier carácter que no sea dígito, así que las letras funcionan como separadores y se descartan; quedan los dígitos. Como la cadena empieza con 'a', el primer elemento es la cadena vacía, por lo que la salida exacta es ' 1 2 3 4 5 6 ' (con un espacio inicial), que corresponde a la opción a. 123456 olvida el espacio que se imprime tras cada parte, y abcdef supone que se conservan las letras en vez de los dígitos.

Fuente: Simulación APX V1 #11 · también aparece en: Simulación APX V4 #20, Java.pdf #56

La respuesta correcta es: 1 2 3 4 5 6

Mi justificación
Al hacer el Split se eliminian las letras y se dejan sólo los números, imprimiendo 123456

Pregunta 11

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

What is the result?

```
1 public class Boxer1{
2     Integer i;
3     int x;
4     public Boxer1(int y){
5         x = i+y;
6         System.out.println(x);
7     }
8     public static void main(String[] args){
9         new Boxer1(new Integer(4));
10    }
11 }
```

Seleccione una:

- a. Compilation fails because of an error in line 5
- b. The value "4" is printed at the command line. ⊗
- c. A NullPointerException occurs at runtime
- d. An IllegalStateException occurs at runtime.
- e. Compilation fails because of an error in line 9
- f. A NumberFormatException occurs at runtime

Justificación: El campo Integer i vale null por defecto. En la línea 5, i + y obliga a desempaquetar (i.intValue()), lo que lanza NullPointerException. La línea 9 compila porque el Integer se desempaqueta a int para el constructor. new Integer(4) está obsoleto desde Java 9, pero solo genera una advertencia, no un error de compilación.

Fuente: Java DE #13 · también aparece en: Java D #22

La respuesta correcta es: A NullPointerException occurs at runtime

Mi justificación
Si Boxer1(x) recibe (Integer x = 4) como parámetro, entonces imprimiría directamente ese 4.

Pregunta 12
Incorrecta
Se puntúa 0,00 sobre 1,00
Marcar pregunta

Nivel: **Avanzado**

What is the result?

```
public class Test {
    public static void main(String[] args) {
        String str = "Hello";
        modify(str);
        System.out.println(str);
        StringBuffer sbf = new StringBuffer("Hello");
        modify(sbf);
        System.out.println(sbf.replace(2, sbf.length(), "o"));
        StringBuilder sbd = new StringBuilder("Hello");
        modify(sbd);
        System.out.println(sbd.reverse());
    }
    private static void modify(String s) {
        s = s.concat(" World");
    }
    private static void modify(StringBuffer sb) {
        sb.append(" World");
    }
    private static void modify(StringBuilder sb) {
        sb.append(" World");
    }
}
```

Seleccione una:

a. Hello Hello World Hello World
b. Hello Hello Hello
c. Hello Heo dlroW olleH
d. Hello World Heo Hello World
e. Hello Heo World Hello ✗

Justificación: String es inmutable: concat crea otro objeto y str sigue valiendo "Hello". StringBuffer y StringBuilder se modifican a través de la referencia: sbf pasa a "Hello World" y replace(2, length, "o") deja "Heo"; sbd pasa a "Hello World" y reverse() da "dlroW olleH". Se imprimen en tres líneas: Hello / Heo / dlroW olleH. El distractor a supone que el String también cambia. NOTA DE FUENTE: en el PDF el código no compilaba (faltaba ';' tras println(str) y se llamaba a modify(...) mientras los métodos se llamaban modifyString, modifyStringBuffer y modifyStringBuilder), y su explicación daba como respuesta «c. Compilation fails», aunque la opción c dice «Hello Heo dlroW olleH» y ninguna opción ofrece «Compilation fails».

Nota sobre el material original: El código del material original no compilaba (faltaba un ';' y los métodos modify tenían otros nombres), y ninguna opción decía «Compilation fails»; se corrigió el código a su intención evidente (tres métodos modify sobrecargados), con el que la opción c es la salida real.

Fuente: Java.pdf #116

La respuesta correcta es: Hello Heo dlroW olleH

Mi justificación
No leí bien pero pensé que siguiendo la secuencia del main se podría obtener fácilmente.

RESULTADO = 7/12 = 5.8